06 POINTERS AND REFERENCES • DETAILED SOLUTIONS

conceptual solutions

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

Detailed Solutions — Conceptual Questions (Q1–Q10)

Solution to Q1 (Pointer Definition vs. Ordinary Variable)

1. **Question Restated**: What is a pointer in C++, and how does its stored value differ from an ordinary `int` variable?
2. **Concept Tested**: Memory addresses vs. direct values (`[basic.compound]`).
3. **Relevant Rule & Reasoning**:
   - An ordinary `int` variable (`int x = 25;`) stores a direct numerical quantity (`25`) in its memory bytes.
   - A pointer (`int* p = &x;`) stores the **virtual memory address** (e.g., `0x7ffeefbff5ac`) where `x` resides in memory.
4. **Common Mistake**: Confusing `p` (the address stored inside `p`) with `*p` (the integer value stored at that address).

Solution to Q2 (`&` and `*` Operators; Why `&42` Fails)

1. **Question Restated**: Explain `&` and `*` , and why `&42` fails to compile.
2. **Concept Tested**: Value categories (`lvalue` vs `prvalue`) and unary operators (`[expr.unary.op]`).
3. **Relevant Rule & Reasoning**:
   - Unary `&` requires an **lvalue** operand (an object with a persistent identity in memory) and returns its address (`T*`).
   - The integer literal `42` is a **prvalue** (pure rvalue) with no addressable storage location, so `&42` is ill-formed (Compilation Error: *lvalue required as unary* `&` *operand*).
   - Unary `*` takes a valid pointer operand (`T*`) and yields an **lvalue** referring to the target object of type `T` .

Solution to Q3 (`nullptr` vs `NULL` and `0`)

1. **Question Restated**: Why prefer `nullptr` over `NULL` or `0` ?
2. **Concept Tested**: Type safety of `std::nullptr_t` (`[lex.nullptr]`).
3. **Relevant Rule & Reasoning**:
   - `0` and `NULL` have integer types (`int` or `long`), which silently convert to integer parameters in overloaded functions (`void f(int)` vs `void f(int*)`).

- `nullptr` has type `std::nullptr_t`, which converts implicitly to any pointer type (`T*`) but **never** to arithmetic/integer types, eliminating overload ambiguity.

Solution to Q4 (Why Pointers Have Distinct Types Despite Equal `sizeof`)

1. **Question Restated**: Why does C++ distinguish `char*`, `int*`, and `double*` if all are 8 bytes on a 64-bit machine?
2. **Concept Tested**: Dereference width/interpretation and scaled pointer arithmetic.
3. **Relevant Rule & Reasoning**:
   - **Dereferencing (`*p`)**: The compiler must know whether to read 1 byte (`char`), 4 bytes (`int`), or 8 bytes (`double`), and how to decode the bit pattern (integer vs. IEEE-754 float).
   - **Pointer Arithmetic (`p + k`)**: Adding `1` to `p` advances the raw byte address by `1 * sizeof(T)` bytes (`+1` for `char*`, `+4` for `int*`, `+8` for `double*`).

Solution to Q5 (`const int* p` vs `int* const p` vs `const int* const p`)

1. **Question Restated**: Differentiate the three `const` pointer forms.
2. **Concept Tested**: Low-level (pointed-to) `const` vs. top-level (pointer) `const`.
3. **Step-by-Step Analysis**:
   - `const int* p`: Pointer to `const int`. `p = &other` is **Legal**; `*p = 100` is a **Compile Error**.
   - `int* const p`: `const` pointer to `int`. `p = &other` is a **Compile Error**; `*p = 100` is **Legal**.
   - `const int* const p`: `const` pointer to `const int`. Both `p = &other` and `*p = 100` are **Compile Errors**.

Solution to Q6 (`void*` Generic Pointer Restrictions)

1. **Question Restated**: What is `void*` and what two operations fail on it?
2. **Concept Tested**: Incomplete type `void` (`[basic.fundamental]`).
3. **Relevant Rule & Reasoning**:
   - `void*` can hold the address of any non-const data object without knowing its concrete type.
   - **Illegal Operation 1**: Direct dereferencing (`*vp`) — fails because `sizeof(void)` and target representation are unknown.
   - **Illegal Operation 2**: Pointer arithmetic (`vp + 1` or `vp++`) — fails in ISO C++ because pointer arithmetic scales by `sizeof(T)`, which is undefined for incomplete type `void`.

Solution to Q7 (Wild vs. Null vs. Dangling Pointer vs. Memory Leak)

1. **Question Restated**: Distinguish wild, null, and dangling pointers and memory leaks.
2. **Step-by-Step Breakdown**:
   - **Wild Pointer**: Uninitialized pointer (`int* p;`) containing arbitrary stack garbage. Cannot be reliably checked with `p ≠ nullptr`!
   - **Null Pointer**: Explicitly initialized to `nullptr`. **Can** be safely checked with `if (p ≠ nullptr)`.

- **Dangling Pointer**: Points to memory whose object lifetime has ended (e.g., after `delete p;` or returning `&local`). Cannot be detected with `p ≠ nullptr` unless `p` was manually reset to `nullptr`.
- **Memory Leak**: Heap memory allocated via `new` whose address is lost before `delete` is called.

Solution to Q8 (Pointers vs. References & Why `int& arr[5]` is Illegal)

1. **Question Restated**: Give six differences between `int*` and `int&`, and explain why arrays of references are forbidden.
2. **Concept Tested**: Reference aliasing rules (`[dcl.ref]`).
3. **Reasoning**:
   - Six differences: (1) Initialization is mandatory for `int&`, optional for `int*`; (2) `int*` can be rebound, `int&` cannot; (3) `int*` can be `nullptr`, `int&` cannot; (4) `int*` needs `*` to dereference, `int&` is automatic; (5) `&p` gives the pointer's own address, `&ref` gives the target's address; (6) `int**` exists, whereas references to references collapse.
   - **Why `int& arr[5]` is illegal**: Array elements must be objects of known size that can be contiguous in memory and addressed via pointer arithmetic (`arr + i`). References are not objects (`[basic.compound]`) and do not have their own distinct addresses.

Solution to Q9 (Precedence vs. Associativity vs. Evaluation Order / Sequencing)

1. **Question Restated**: Distinguish precedence, associativity, and evaluation order, and explain why `*p++ + *p++` is UB.
2. **Concept Tested**: Parse tree construction vs. runtime execution sequencing (`[intro.execution]`).
3. **Reasoning**:
   - **Precedence** dictates that `*p++` parses as `*(p++)`.
   - **Associativity** dictates that `++*p` parses right-to-left as `++(*p)`.
   - **Evaluation Order (Sequencing)** governs *when* the side effects of left and right operands of `+` occur at runtime. Because binary `+` leaves its left and right operands **unsequenced**, both `p++` side effects modify scalar `p` without a sequencing point between them, resulting in **Undefined Behavior**.

Solution to Q10 (Smart Pointers & Breaking Cycles with `std::weak_ptr`)

1. **Question Restated**: Compare raw pointers with `unique_ptr`, `shared_ptr`, and `weak_ptr`.
2. **Concept Tested**: RAII ownership and reference counting (`<memory>`).
3. **Reasoning**:
   - `std::unique_ptr<T>`: Exclusive ownership; zero overhead; non-copyable, movable.
   - `std::shared_ptr<T>`: Shared ownership via atomic reference counting.
   - `std::weak_ptr<T>`: If Object A holds a `shared_ptr<B>` and Object B holds a `shared_ptr<A>`, their reference counts never drop to `0` when leaving scope, causing a memory leak. Replacing one link with `std::weak_ptr` observes the target without incrementing the strong reference count, allowing deterministic destruction.